An Approach Towards Building a Smart Lift System Using Machine Learning

Shanouf Ansari[1]
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, India
mohammed.ansari@spit.ac.in

Allen Andrew[2]
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, India
allen.andrew@spit.ac.in

Sudhir Dhage[3]
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, India
sudhir_dhage@spit.ac.in

***Abstract*—Lift systems have traditionally operated based on pre-programmed schedules and static behavior, limiting their ability to adapt to real-time user demands. This lack of real-time adaptability results in inefficient energy usage, as lifts continue to operate on fixed routes regardless of fluctuations in user patterns. Moreover, the rigid system structure leads to longer wait times, especially during peak demand periods, where multiple users require service simultaneously. In buildings with multiple lifts, the problem is exacerbated, as the coordination between lifts becomes more complex, further diminishing operational efficiency and increasing overall energy consumption. Additionally, optimizing lift routes and energy usage remains a significant challenge, as current systems fail to dynamically adjust based on real-time conditions, such as building occupancy or time of day. This research aims to explore the potential of machine learning (ML)-based smart lift systems that can address these issues by enabling real-time adaptability. Our approach will leverage advanced ML algorithms to analyze user patterns, predict traffic flow, and optimize lift routes dynamically. By continuously learning from historical and real-time data, the system will be able to predict peak times, adjust lift operations, and reduce energy consumption while maintaining efficient service. We believe that this approach can significantly reduce wait times, improve energy efficiency, and enhance user experience, leading to smarter, more sustainable lift systems in modern buildings. The research will contribute to the design of intelligent systems capable of optimizing both user satisfaction and operational costs.**

***Index Terms*—Lift Call Prediction, Energy Efficiency, Smart Elevator Systems, Real-time Adaptability**

I. Introduction

Traditional lift systems operate based on pre-programmed schedules and fixed operational parameters, which do not account for real-time user demands or building occupancy patterns. As a result, these systems often suffer from inefficiencies, such as unnecessary energy consumption and longer wait times for users, particularly during peak demand periods. When multiple lifts are present in a building, the lack of coordination between them further exacerbates these issues, leading to suboptimal routes and poor energy management. This static operation fails to respond dynamically to the constantly changing patterns of lift usage, making it increasingly difficult to optimize both energy usage and service delivery.

Recent advancements in machine learning (ML) offer an opportunity to revolutionize lift systems by enabling them to adapt in real-time to user behavior and environmental conditions. ML-based smart lift systems have the potential to analyze patterns of lift usage, predict demand surges, and optimize routing and energy consumption accordingly. By learning from historical data and continuously adapting to real-time inputs, these systems can offer improved energy efficiency, shorter wait times, and better coordination among multiple lifts. This research seeks to explore how ML can be applied to enhance the performance of lift systems, with the goal of reducing energy waste, improving user experience, and contributing to more sustainable and efficient building operations.

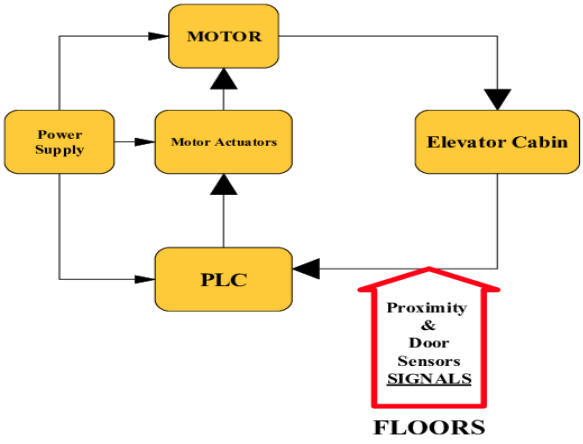


Fig. 1: A Generic Lift System

II. Literature Survey

"Rehabilitation of an Old Traditional Elevator Based on PLC Techniques" [1] , proposed by Thabet *et al.* explored the rehabilitation of an outdated traditional elevator system using programmable logic controller (PLC) techniques. Their study, published in the Wasit Journal of Engineering Sciences, focused on the integration of PLCs to improve the functionality and reliability of older elevator systems. This work emphasizes the significance of retrofitting existing elevator infrastructure with modern technologies to extend their operational life while reducing maintenance costs and improving safety. The use of PLCs offers enhanced control and monitoring capabilities,

addressing the limitations of older elevator systems that often lack automation and fail-safe mechanisms.

The paper titled "Simplified Energy Calculations for Lifts Based on ISO/DIS 25745-2" [2] by Barney and Lorente presented a simplified approach to energy calculations for lifts in line with the ISO/DIS 25745-2 standard at the 3rd Symposium on Lift and Escalator Technologies. Their paper focused on developing a more accessible method for assessing the energy consumption of elevators, aimed at improving the efficiency and sustainability of lift systems. The authors provide valuable insights into the energy-saving potential of lifts through simplified modeling techniques, ensuring that elevator designs comply with modern environmental and energy-efficiency standards.

"Lift Power Consumption" [3] by Al-Sharif analyzed the power consumption patterns of lift systems in Elevator World. The paper explores various factors influencing the energy efficiency of lifts, including motor performance, load capacity, and operational frequency. Al-Sharif's research was pivotal in understanding the energy demands of elevator systems and laid the groundwork for future innovations in reducing lift power consumption. His findings remain relevant for designing more energy-efficient systems, particularly in the context of smart elevators and modern building automation.

The paper "Connected Smart Elevator Systems for Smart Power and Time Saving" [4] proposed by Rashed *et al.* investigated the potential of connected smart elevator systems for optimizing power consumption and reducing wait times in buildings. Published in Scientific Reports, their research proposes a novel approach that integrates smart technologies, such as IoT and machine learning, to enhance the operational efficiency of elevator systems. By utilizing real-time data and predictive algorithms, the proposed smart system not only reduces energy consumption but also improves user experience by minimizing waiting times, making it a significant advancement in building automation.

The paper titled "Long Short-Term Memory" [5] proposed by Hochreiter and Schmidhuber's seminal 1997 paper on Long Short-Term Memory (LSTM) networks, revisited in 2020, remains a cornerstone of deep learning research. LSTM, a type of recurrent neural network (RNN), addresses the vanishing gradient problem in traditional RNNs, making it highly effective for sequence prediction tasks. This breakthrough in machine learning has had widespread applications, including time series forecasting and natural language processing. LSTM's ability to capture long-range dependencies in data makes it particularly useful for modeling sequential patterns, such as those found in elevator operation and predictive maintenance.

The paper "LSTM-Based Deep Learning for Sentiment Analysis" [6] by Saha and Senapati focuses on the use of LSTM networks for sentiment analysis, applied to both English and Spanish datasets. Their study, presented at the International Conference on Computational Performance Evaluation, highlights the power of LSTM in understanding and classifying emotional tones in text data. By employing deep learning techniques, the authors demonstrate the potential of LSTM models to process and analyze large volumes of unstructured textual data. This research is important for advancing natural language processing techniques and could be extended to user feedback analysis in smart lift systems.

The paper titled "Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM)" [7] presented by Sherstinsky provides a detailed explanation of Recurrent Neural Networks (RNN) and LSTM networks, focusing on their theoretical foundations and practical applications. Published in Physica D: Nonlinear Phenomena, the paper explains how LSTMs are particularly adept at learning from sequential data due to their ability to maintain long-term memory. This characteristic makes LSTMs suitable for tasks such as speech recognition, time series prediction, and, notably, optimizing systems like smart elevators, where past performance data can predict future operations and maintenance needs.

"Recurrent Neural Networks: An Embedded Computing Perspective" [8] proposed by Rezk *et al.* explores the application of recurrent neural networks (RNNs) from an embedded computing perspective. Published in IEEE Access, the study discusses how RNNs, particularly LSTMs, are increasingly being integrated into embedded systems for real-time applications. This work is relevant for the development of smart elevator systems, where embedded computing platforms must process sensor data and make real-time decisions about elevator operation, maintenance, and optimization.

The paper "Smart Elevator Systems" [9] by Al-Kodmany provides an extensive review of smart elevator systems in his paper published in the Journal of Mechanical Materials and Mechanics Research. The study discusses the integration of advanced technologies like IoT, AI, and machine learning in the operation of modern elevator systems. Smart elevators, according to Al-Kodmany, offer enhanced functionality, including predictive maintenance, energy optimization, and personalized user experiences. This research is key for the ongoing development of elevators that are more intelligent and efficient, aligning with the growing trend of building automation.

The paper "Towards Personalized Elevator Travel with Smart Elevator System" [10] by Robal *et al.* explores the concept of personalized elevator travel in their paper, published in the Baltic Journal of Modern Computing. The authors discuss how machine learning and smart algorithms can be used to tailor elevator travel based on user preferences, such as travel times and destinations. This personalized approach improves user satisfaction and operational efficiency by reducing waiting times and energy consumption. Their work is a significant step towards the next generation of smart elevator systems that are adaptable to individual needs.

III. Research and Methodologies

A. Interaction Schematic

The architecture represents a workflow designed to research and develop a smart lift system using machine learning and simulation techniques. The process begins with a Seed, which

acts as an initial input to the system. This seed helps simulate lift call data that mimics real-world behavior, including noisy periodicity and peak-hour demands. This synthesized data is crucial as it replicates the random yet structured patterns of lift usage, providing a realistic foundation for analysis and model training. The simulation of lift calls ensures the generated data reflects various challenges, such as irregular call distributions, enabling robust experimentation.

Once the synthetic lift call data is generated, it feeds into two key processes. First, it enters the Lift Simulation Model, where decisions are made regarding lift operations. The simulation model serves as an environment that replicates real-time lift behavior based on the input data. Simultaneously, the generated data undergoes Input Pre-processing and Sequencing, which organizes and structures the data for machine learning. This step ensures that the data is cleaned, properly formatted, and sequenced for training predictive models. Proper preprocessing allows the model to learn effectively from the simulated lift calls and make accurate predictions.

The ML-Based Predictive Model is a critical component of the architecture, where machine learning techniques are applied to predict future lift calls and optimize decision-making. The preprocessed data is used to train this predictive model, enabling it to identify patterns and anticipate future demands. The trained model interacts with the Decision component within the lift simulation environment. This interaction forms a feedback loop, where the decisions made based on the predictions are continuously evaluated and refined. The ML model learns from these decisions, improving its accuracy over time as it adapts to the simulated environment.

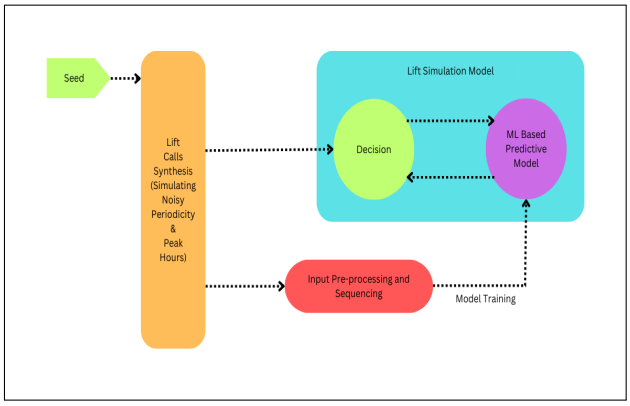


Fig. 2: Basic Simulation Interaction Schematic

Throughout the project, this architecture supports iterative research by combining synthetic data generation, preprocessing, simulation, and machine learning. The feedback loop between the Decision and ML-Based Predictive Model components ensures that the model is evaluated and updated based on realistic scenarios. This structure enables researchers to test different machine learning algorithms, experiment with noisy and peak-hour data, and fine-tune the predictive model for improved performance. By simulating a variety of conditions and incorporating real-time decision-making, the architecture facilitates the development of a smart, efficient lift system capable of handling dynamic, real-world challenges.

B. Lift Calls Synthesis

The Lift Call Synthesis module has two distinct variants designed to generate realistic lift usage patterns: one focuses on mimicking periodicity, while the other simulates peak hours. The periodicity variant captures regular patterns of lift calls throughout the day, such as morning, afternoon, and evening usage cycles, reflecting predictable user behavior in residential or office buildings. This ensures the model accounts for consistent, recurring lift demands over time. In contrast, the peak hour variant introduces bursts of activity during specific time windows, such as rush hours in the morning or evening, where lift usage is significantly higher and less evenly distributed. This variant mimics real-world congestion scenarios, where sudden surges in calls can stress the lift system. Together, these two variants enable comprehensive data synthesis, balancing regular and chaotic lift usage patterns, which is critical for training predictive models to handle both predictable and extreme conditions effectively.

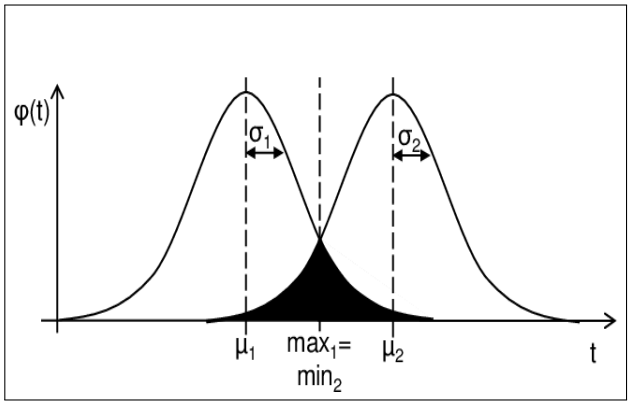


Fig. 3: Gaussian Components as Peak Call Hours

To create periodicity-mimicking lift calls, a complex composition of trigonometric functions was used to replicate regular, cyclic patterns observed in real-world lift usage. Sine and cosine functions were employed to simulate daily periodicity, such as morning, afternoon, and evening cycles. These functions operate on different time scales, with varying amplitudes and frequencies to reflect the intensity and duration of lift calls across a 24-hour period. By combining multiple trigonometric components, each representing distinct usage patterns, the synthesis process could generate a smooth, recurring baseline that captures the rhythm of typical lift demands. For instance, a high-frequency sine wave might represent short, regular calls, while a low-frequency wave models broader trends like increased activity during specific times of the day.

The visual above describes a mathematical model for synthesizing lift calls, which is crucial for simulating and optimizing elevator systems. The model aims to capture the stochastic nature of passenger arrivals and their destination

floor selections. This section explains the components of this model.

1) Model Description: The core of the model is the equation:

$$C(f,t) = I(t) \cdot P(f) + \epsilon(f,t)$$

Where:

- $C(f,t)$: The expected number of lift calls originating from floor f at time t.
- $I(t)$: A time-based intensity function that captures the overall arrival rate of passengers.
- $P(f)$: The floor importance factor, representing the probability that a call originates from floor f.
- $\epsilon(f,t)$: A Gaussian noise term that introduces randomness to the model, accounting for individual variations in call patterns.

2) Time-Based Intensity Function, $I(t)$: The intensity function $I(t)$ is designed to model the non-uniform arrival rate of passengers throughout the day. It incorporates two key aspects: peak hours and periodic fluctuations. The image provides the following equation for $I(t)$:

$$I(t) = \alpha_1 \cdot e^{-\frac{(t-\mu_1)^2}{2\sigma_1^2}} + \alpha_2 \cdot e^{-\frac{(t-\mu_2)^2}{2\sigma_2^2}} + \delta \cdot \sin\left(\frac{\pi t}{24}\right)$$

Let's break down this equation:

- **2-Peak Simulation:** The first two terms represent the peak hours. These are modeled as Gaussian distributions:
  - α_1 and α_2: Amplitudes of the two Gaussian peaks, representing the height of the peaks (i.e., the intensity of calls during the peak hours).
  - μ_1 and μ_2: Mean values of the two Gaussian distributions, representing the times at which the peak hours occur. For example, μ_1 might represent 8:00 AM, and μ_2 might represent 5:00 PM.
  - σ_1 and σ_2: Standard deviations of the two Gaussian distributions, representing the width of the peaks (i.e., how spread out the peak hours are).
- **Periodicity Simulation:** The third term models the daily periodic fluctuation of passenger arrivals:
  - δ: Amplitude of the sinusoidal term, representing the magnitude of the daily fluctuation.
  - $\sin\left(\frac{\pi t}{24}\right)$: A sinusoidal function with a period of 48 hours. Since time *t* is likely measured in hours, dividing by 24 and multiplying by π scales the sine function to capture the daily cycle (note that the period is 48 because the sine function completes half a cycle in 24 hours).

In summary, $I(t)$ combines two Gaussian peaks to represent morning and evening rush hours with a sinusoidal term to capture the general daily pattern of elevator usage.

3) Floor Importance, $P(f)$: The term $P(f)$ represents the probability that a lift call originates from floor f. It is a crucial factor in distributing the total number of calls across different floors. Floors with higher $P(f)$ values are expected to generate more calls. The values of $P(f)$ are typically determined empirically, based on data collected from real-world elevator systems. For example:

- The ground floor might have a high $P(f)$ because it's the entry/exit point.
- Office floors in a commercial building might have higher $P(f)$ values during work hours.
- Residential floors in an apartment building might have more uniform $P(f)$ values.

The sum of $P(f)$ over all floors should equal 1:

$$\sum_{f=1}^{F} P(f) = 1$$

where F is the total number of floors.

4) Gaussian Noise, $\epsilon(f,t)$: The term $\epsilon(f,t)$ represents Gaussian noise. It is added to the model to account for the inherent randomness in passenger arrivals. Even with the intensity function and floor importance factors, the actual number of calls at any given time can vary. The Gaussian noise term captures this variation.

- It is assumed that $\epsilon(f,t)$ follows a normal (Gaussian) distribution with a mean of 0 and a variance $\sigma^2(f,t)$, which may or may not depend on the floor and time.
- A mean of 0 implies that, on average, the noise does not systematically increase or decrease the number of calls.
- The variance $\sigma^2(f,t)$ determines the magnitude of the random fluctuations. A larger variance indicates more variability in the number of calls. The variance can be constant, or it can vary with floor (f) or time (t) to reflect different levels of variability. For example, the variance might be higher during peak hours.

C. Lift Simulation Software

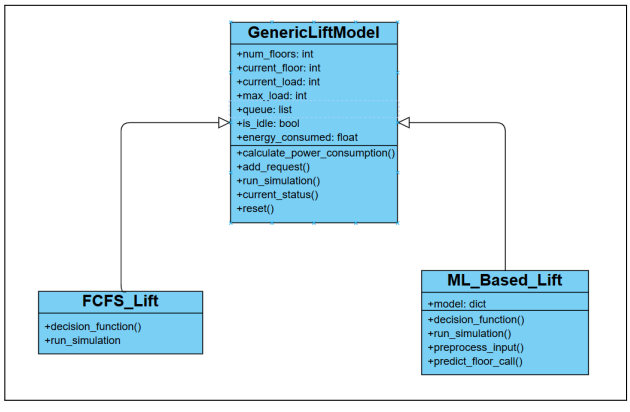


Fig. 4: Basic Class Diagram of the Simulation Entities

To boost the research on smart lift systems, a dedicated simulation software was developed, offering a controlled en-

vironment to rigorously test models and monitor power consumption. The software was designed to replicate a multi-floor building environment, complete with simulated lift operations, user behavior, and varying traffic intensities. By incorporating synthetic lift call data, the software could simulate both periodic and peak-hour scenarios, providing diverse and realistic inputs for testing machine learning models. The simulation environment enabled researchers to tweak key parameters such as the number of floors, lifts, and call frequencies, allowing for customized experiments. This adaptability ensured that the models could be evaluated under a wide range of conditions, from low traffic to extreme congestion, while measuring their efficiency and response times in real-time.

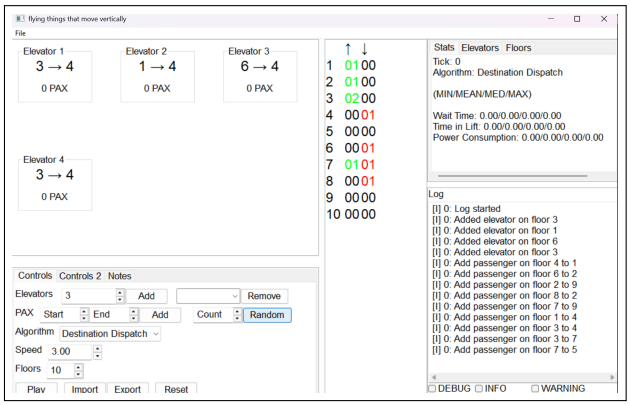


Fig. 5: Simulation Software in Action

A critical feature of the simulation software was its ability to monitor power consumption during lift operations. Power usage was tracked based on parameters like the number of floors traveled, idle time, and the frequency of lift movements. By logging these metrics, the software could evaluate how effectively the predictive models reduced unnecessary lift operations and optimized call handling. For instance, the software assessed whether the model minimized energy-intensive movements, such as lifts traveling empty or making redundant stops. By simulating energy usage across different strategies, researchers could compare model performance in terms of both operational efficiency and power savings, aligning with sustainability goals for smart building systems.

The simulation software also enabled iterative testing and model refinement by integrating a feedback loop. After running simulations, performance metrics such as energy consumption, waiting times, and decision accuracy were analyzed to identify areas for improvement. Machine learning models could then be retrained with updated parameters or fine-tuned algorithms, ensuring continuous enhancement. The software's ability to replicate real-world complexities, such as noisy lift calls, variable user demands, and peak-hour surges, provided a robust platform for experimentation. Researchers could also test advanced control strategies, like load balancing or predictive scheduling, to determine their impact on power efficiency and user experience. This iterative approach not only accelerated the research process but also provided valuable insights into designing lift systems that are both energy-efficient and capable of handling dynamic usage patterns effectively.

D. Proposed Models

Initially, experiments were conducted with a single-lift system to simplify the environment and gain a better understanding of machine learning (ML) integration for predictive scheduling. By focusing on a single lift, researchers could isolate key challenges such as learning periodic patterns, handling noisy call data, and optimizing lift movement decisions. This simplified framework enabled easier debugging, model validation, and analysis of ML behavior when exposed to synthesized lift call data. It provided a foundational understanding of how predictive models could anticipate lift calls and minimize unnecessary movements to improve efficiency. Once the single-lift experiments yielded promising results, the scope was expanded to a multi-lift system, where multiple lifts operated simultaneously, requiring advanced coordination and scheduling strategies.

1) LSTM Model: The first model proposed utilizes a sequential architecture built with Long Short-Term Memory (LSTM) layers to predict lift call sequences. The input to the model consists of an input sequence and a random seed to initialize the training process. The first layer in the architecture is an LSTM layer with 128 units, responsible for capturing long-term dependencies and sequential patterns within the input data. LSTM layers are ideal for time-series prediction tasks, as they are designed to retain information over longer sequences without suffering from vanishing gradient issues. Following the first LSTM layer, a Dropout layer with a rate of 0.2 is introduced to prevent overfitting by randomly setting a fraction of the input units to zero during training. This ensures the model generalizes well to unseen data.

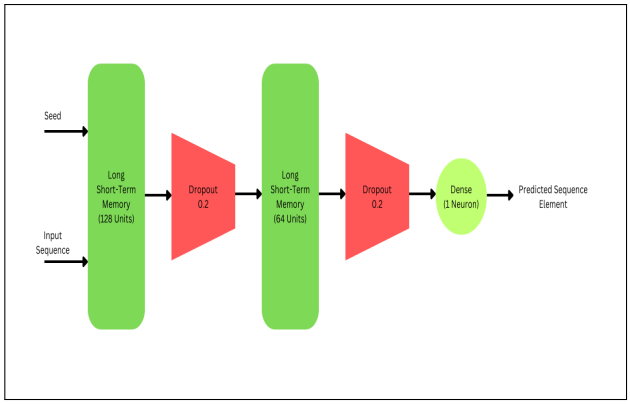


Fig. 6: Pure LSTM-Based Architecture

The second stage of the architecture involves another LSTM layer with 64 units, which further processes the output from the previous layer and refines the learned temporal features. Similar to the earlier stage, a Dropout layer with a rate of 0.2 is added after the second LSTM layer to improve regularization. The final output is passed through a Dense layer with a single neuron, which generates the predicted sequence

element. The dense layer serves as the output layer and applies a linear activation function to map the LSTM outputs to the target prediction value. This architecture is simple yet effective for time-series tasks, as it leverages multiple LSTM layers to extract meaningful sequential features while maintaining robustness through dropout regularization.

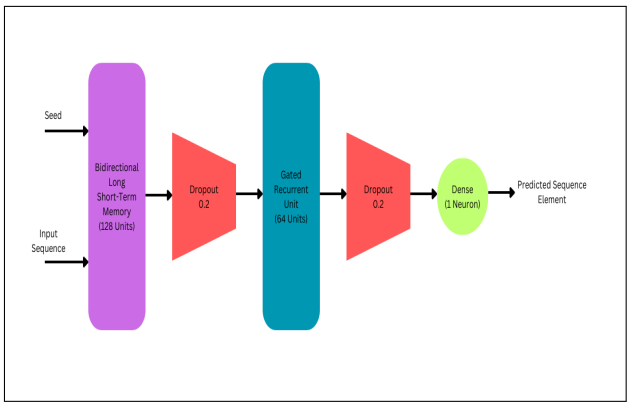


Fig. 7: Hybrid Bi-LSTM + GRU Architecture

2) Bi-LSTM + GRU Model: The second model proposed combines Bidirectional Long Short-Term Memory (Bi-LSTM) and Gated Recurrent Unit (GRU) layers to enhance the ability to capture complex temporal patterns in lift call sequences. The input consists of an input sequence and a seed, which initialize the model for training. The first layer is a Bidirectional LSTM with 128 units, which processes the input data in both forward and backward directions, enabling the model to capture past and future dependencies in the sequence. This is particularly beneficial for time-series tasks where understanding relationships in both directions improves prediction accuracy. Following this, a Dropout layer with a rate of 0.2 is applied to reduce overfitting by randomly dropping a fraction of the connections during training.

The output of the Bidirectional LSTM is passed to a Gated Recurrent Unit (GRU) layer with 64 units. GRU layers, being computationally efficient variants of LSTMs, help refine the sequential features learned so far by managing information flow through their gating mechanisms. Another Dropout layer with a rate of 0.2 follows the GRU to maintain generalization. Finally, a Dense layer with a single neuron outputs the predicted sequence element using a linear activation function. This hybrid architecture combines the strengths of Bi-LSTM and GRU, ensuring robust performance in capturing both short- and long-term dependencies in the data.

3) Active Learning RNN: Active learning has emerged as a critical paradigm in machine learning where the model is allowed to interactively query a source of information (typically a human or environment) to obtain the desired outputs at new data points. This is especially useful in domains where labeling data is expensive or time-consuming. Recurrent Neural Networks (RNNs), known for their capacity to process sequential data, can benefit significantly from active learning strategies. In the context of elevator systems, predicting lift

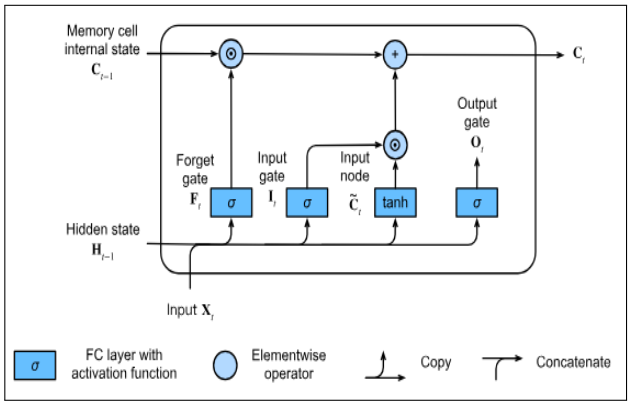


Fig. 8: Active Learning RNN Architecture

calls is inherently a temporal problem influenced by numerous factors such as time of day, day of the week, building usage patterns, and even special events. Active learning allows the RNN to selectively focus on the most informative samples—those that improve model performance the most when added to the training set. Consider a scenario where lift calls spike during morning hours in an office building; an RNN trained passively might underperform if it overfits to less active periods. However, by actively querying ambiguous or surprising call sequences, the RNN can learn evolving patterns more effectively. Mathematically, given an unlabeled sequence set $U = \{x_1, x_2, ..., x_n\}$, the model selects a subset $S \subset U$ using a query strategy Q such that the expected error reduction is maximized:

$$S = \arg \max_{S' \subset U, |S'|=k} \mathbb{E}[\Delta \mathcal{L}(S')],$$

where $\Delta \mathcal{L}(S')$ is the reduction in loss after adding S' to the training set.

RNNs, especially advanced variants like LSTMs and GRUs, are well suited for lift call prediction due to their ability to capture long-term dependencies. Lift calls are typically represented as time-series data, where each timestep includes features like floor number, time, call type (up/down), and even user metadata in smart systems. Active learning enhances the RNN's utility in such contexts by dynamically updating the training distribution as new patterns emerge. For instance, when a new company moves into a building's top floors, lift usage patterns may change abruptly. Traditional supervised training would require manual relabeling or re-collection of large datasets to adapt. An active RNN model, however, can flag uncertain or mispredicted sequences for labeling, facilitating faster adaptation. Let h_t be the hidden state of the RNN at time t, then the update is:

$$h_t = f(h_{t-1}, x_t),$$

where f is the recurrence function, such as in GRUs or LSTMs. The active model also maintains an uncertainty score $u_t = \text{Var}(\hat{y}_t)$, which it uses to query the oracle for labels when u_t exceeds a certain threshold τ. This way, lift systems

become more responsive and energy-efficient by anticipating demand patterns accurately.

In high-rise buildings with multiple lifts and high traffic, understanding the temporal distribution of calls becomes crucial. The key to making predictions that evolve with time is not merely memorizing patterns but adapting to their change. Active learning mechanisms embedded in RNNs help create a feedback loop between prediction and data acquisition. For instance, during lunch hours, certain floors may experience heavier traffic. The model might initially struggle with irregular patterns—such as staggered break schedules—but can improve over time by querying and retraining on edge cases. Let $\mathcal{D}_t$ represent the labeled dataset at time t, and $\mathcal{U}_t$ the unlabeled pool. Then, over successive intervals, the model updates via:

$$\mathcal{D}_{t+1} = \mathcal{D}_t \cup Q(\mathcal{U}_t, \theta_t),$$

where θ_t denotes the current model parameters and Q is the querying function. The resultant active RNN can provide near real-time lift call predictions with high accuracy, adapting quickly to evolving usage without retraining from scratch. This is particularly powerful in smart building systems where contextual data like occupancy sensors or calendar events can further inform active selection strategies. Integrating such side information into the RNN input x_t enriches the model's representation space and improves long-term forecasting of call densities and queue lengths.

Beyond efficiency and responsiveness, the application of active learning RNN models to lift call prediction has systemic benefits. Optimized prediction allows better load balancing among lifts, reducing average waiting time and energy consumption. From an operational standpoint, it reduces maintenance costs by preventing overuse of specific elevators due to suboptimal dispatching. Furthermore, in emergency situations—such as evacuations or system failures—having an adaptive model capable of rapidly recalibrating to atypical patterns becomes a critical asset. The model's real-time adaptation capacity hinges on its ability to exploit high-uncertainty regions effectively. A Bayesian RNN framework can be employed for this purpose, where posterior predictive uncertainty guides sampling:

$$p(y_t|x_{1:t}) = \int p(y_t|x_{1:t}, \theta) p(\theta|\mathcal{D}_t)\, d\theta.$$

Sampling from this posterior allows the model to make informed queries that yield the most value. Importantly, active learning reduces data annotation costs by focusing human effort only where it counts. When deployed at scale, such systems can collect data across multiple buildings, generalize learned representations, and continuously improve via federated learning. Thus, active learning RNNs form the backbone of a sustainable, intelligent vertical transportation system.

IV. Result and Analysis

A. Active Learning RNN Performance

In the context of time-series modeling for lift call prediction using recurrent neural networks (RNNs) augmented with active learning strategies, a notable pattern emerged when contrasting models trained with different window sizes of historical data. Specifically, a model trained with a shorter historical window—300 lift calls—exhibited a markedly accurate predictive trend that closely aligned with the actual call pattern observed during testing. The trend similarity between predicted and actual lift calls was not just visually compelling but was also reflected quantitatively through lower root mean square error (RMSE) and higher dynamic time warping (DTW) correlation. The success of this model lies in its ability to capture the most relevant and recent behavior patterns without being diluted by distant historical noise. Active learning played a crucial role here by selectively querying instances that offered the highest potential information gain, thus avoiding redundancy and emphasizing variability and novelty in lift usage patterns. The compact input size helped the RNN focus on localized temporal structures such as morning rush hours, lunchtime peaks, or end-of-day declines, all of which show relatively high consistency on a day-to-day basis. Therefore, the model could efficiently generalize such patterns without overfitting, leading to superior alignment with real-time system behavior.

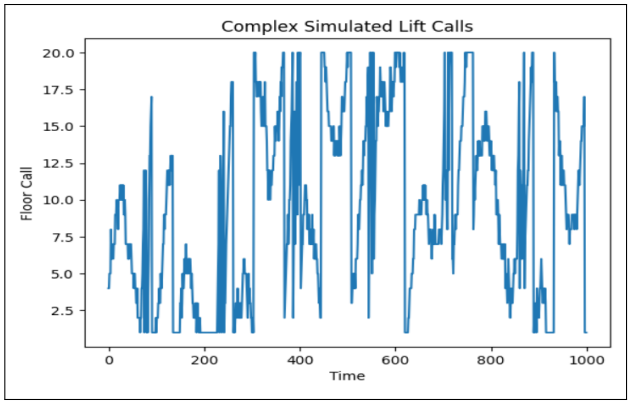


Fig. 9: Synthesized Lift Calls Indicative of Periodicity



Fig. 10: Synthesized Lift Calls Indicative of Peak Hours

On the contrary, a model trained using a much larger input window—specifically incorporating data from the pre-

vious 1000 lift calls—demonstrated significantly degraded performance in both trend prediction and overall accuracy. Despite having access to a broader context, this model suffered from excessive temporal complexity, leading to a failure to discriminate between currently relevant and outdated lift usage patterns. This overabundance of data caused the RNN to spread its focus thinly across too many historical contexts, many of which had no predictive relevance for the current behavior of the lift system. The predicted call patterns generated by this model often deviated significantly from the actual calls, especially during transitional periods such as shift changes or unscheduled usage spikes. This phenomenon can be attributed to the RNN's diminished sensitivity to immediate changes, as it was encumbered by legacy behaviors that no longer represented the active dynamics of the building. Active learning could not sufficiently compensate for this, as the model had already become structurally biased toward long-term dependencies. Thus, despite an abundance of training data, the accuracy and trend fidelity suffered, highlighting that more data does not necessarily equate to better performance when it introduces confounding temporal signals.

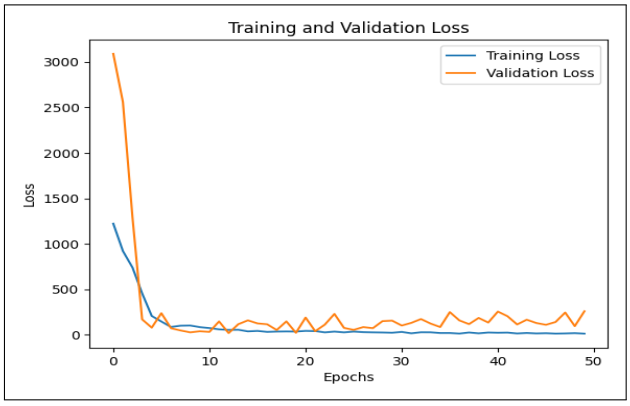


Fig. 11: Training and Validation Losses' Trend

Another critical insight from these experiments is the role of model capacity and optimization behavior under different active learning schemes. With the 300-call window model, the RNN architecture—LSTM layers with carefully tuned hidden units and dropout regularization—remained lean and efficient. Active learning further refined this setup by continuously selecting the most informative samples for retraining, ensuring that model updates were both data-efficient and targeted. This created a feedback loop in which the model continuously improved its short-term generalization capacity without accumulating unnecessary temporal baggage. In contrast, the 1000-call model, despite employing a similar RNN architecture, required more computational resources for each training iteration due to the longer input sequences. The increased sequence length also resulted in gradient vanishing problems during backpropagation through time, which compounded the model's inability to adapt quickly to new trends. In essence, while both models were built on similar RNN foundations, the data selection and historical windowing strategies created vastly different learning dynamics, with the shorter window model clearly outperforming in both predictive sharpness and trend adherence.

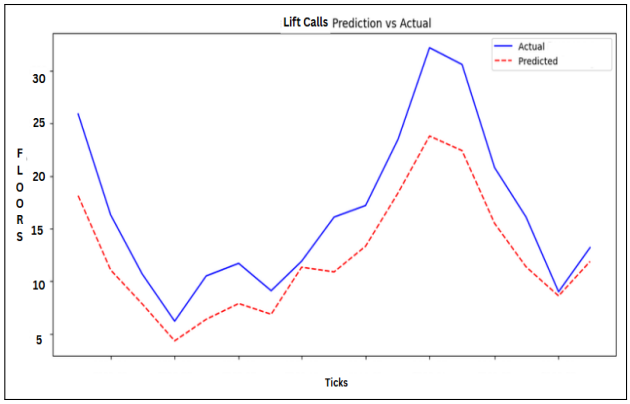


Fig. 12: Lift Calls' Trend Prediction based on Previous 300 Calls

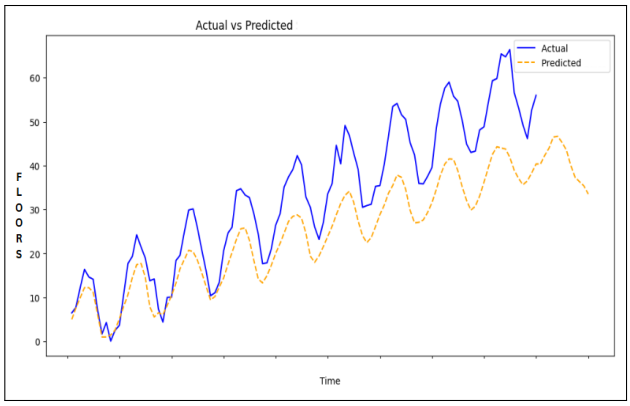


Fig. 13: Lift Calls' Trend Prediction based on Previous 1000 Calls

Finally, from a systems perspective, these findings underscore the critical need for balance between historical context and recency in time-series modeling. While RNNs are theoretically capable of capturing long-term dependencies, in practice, their performance in real-time, event-driven environments like elevator systems is heavily constrained by both computational feasibility and the risk of overfitting to irrelevant data. The 300-call model's effectiveness demonstrates that optimal predictive performance does not demand exhaustive data, but rather strategically sampled, relevant, and recent data. Active learning shines in this scenario by allowing the model to remain focused and continuously calibrated to current trends. This experiment suggests that future improvements in predictive systems for lifts and similar applications should prioritize not just model architecture, but the intelligent design of data windows and active querying strategies. By combining compact sequence lengths with robust uncertainty sampling, practitioners can build systems that are not only accurate but

also agile, resource-efficient, and better suited for deployment in dynamic operational contexts.

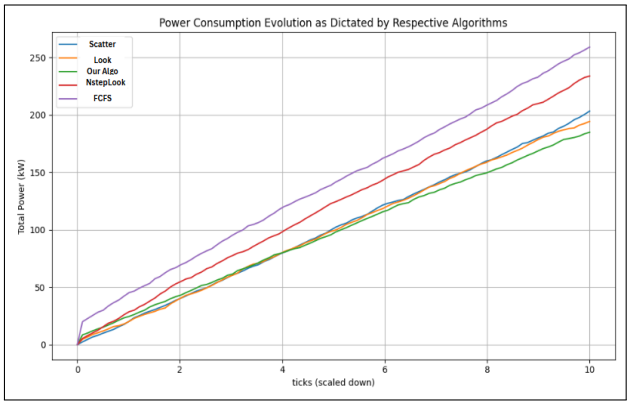


Fig. 14: Lift Power Consumption Evolution

V. Conclusion

Researching machine learning-based lift systems has revealed the transformative potential of artificial intelligence in optimizing building infrastructure performance. Traditional lift control methods, which often rely on fixed scheduling, predefined rules, or basic heuristics, lack the flexibility to effectively adapt to varying and unpredictable usage patterns throughout the day. In contrast, machine learning models—particularly advanced architectures like Long Short-Term Memory (LSTM) and hybrid approaches such as Bi-LSTM combined with Gated Recurrent Units (GRU)—demonstrate a strong capacity to analyze historical usage data and accurately forecast future lift demand. This predictive ability enables real-time, data-driven decision-making, allowing lift systems to anticipate peak traffic periods and adjust operations accordingly. As a result, significant gains have been observed in energy efficiency, as the system reduces idle running and minimizes unnecessary lift movements. Additionally, load balancing across multiple elevators becomes more efficient, leading to shorter wait times and smoother service for building occupants. These improvements not only highlight the practicality of implementing ML-driven solutions in real-world environments but also emphasize their potential to support the development of intelligent, responsive, and energy-conscious smart buildings. The research confirms that machine learning is not merely a conceptual advancement but a viable tool for redefining lift system management.

VI. Future Scope

Looking ahead, there is significant potential to expand this research through the development of a comprehensive simulator interface that offers a visual and interactive representation of the lift system's operation. Such a simulator would not only help validate the underlying model in near-real-world conditions but also provide a powerful tool for experimenting with and refining control algorithms. By simulating various operational scenarios, this interface can offer deeper insights into system dynamics and highlight areas for improvement. Furthermore, incorporating real-world data from actual lift systems could elevate the model's applicability, though it introduces challenges such as managing noisy or incomplete data and ensuring compatibility with existing building infrastructure. Addressing these challenges may involve the integration of preprocessing techniques and robust data pipelines. To further enhance adaptability, active learning models could be employed, enabling the system to learn from incoming data continuously. This approach allows the model to evolve in response to changing usage patterns, environmental conditions, or user behavior, ensuring sustained performance over time. Ultimately, these advancements would improve the model's precision and resilience, contributing to more efficient, intelligent lift systems and demonstrating the broader capabilities of machine learning in complex, real-world optimization problems.

References

[1] Thabet, Hassaan Jameel, Ibraheem Hasan, Ismail. (2018). Rehabilitation of an Old Traditional Elevator Based on PLC Techniques. Wasit Journal of Engineering Sciences. 6. 76-89. 10.31185/ejuow.Vol6.Iss3.106.

[2] Gina Barney, Ana Lorente. Simplified Energy calculations for Lifts Based on ISO/DIS 25745-2. 3rd Symposium on Lift and Escalator Technologies.

[3] Al-Sharif, Lutfi. (1996). Lift Power Consumption. Elevator World. 44. 85-87.

[4] Rashed, A.N.Z., Yarrarapu, M., Prabu, R.T. et al. Connected smart elevator systems for smart power and time saving. Sci Rep 14, 19330 (2024). https://doi.org/10.1038/s41598-024-69173-1.

[5] Hochreiter, Sepp Schmidhuber, Jürgen. (2020). Long Short-term Memory. Neural computation. 9. 1735-80. 10.1162/neco.1997.9.8.1735.

[6] B. Nath Saha and A. Senapati, "Long Short Term Memory (LSTM) based Deep Learning for Sentiment Analysis of English and Spanish Data," 2020 International Conference on Computational Performance Evaluation (ComPE), Shillong, India, 2020, pp. 442-446, doi: 10.1109/ComPE49325.2020.9200054.

[7] Alex Sherstinsky, Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) network, Physica D: Nonlinear Phenomena, Volume 404, 2020, 132306, ISSN 0167-2789.

[8] N. M. Rezk, M. Purnaprajna, T. Nordström and Z. Ul-Abdin, "Recurrent Neural Networks: An Embedded Computing Perspective," in IEEE Access, vol. 8, pp. 57967-57996, 2020, doi: 10.1109/ACCESS.2020.2982416.

[9] Al-Kodmany, Kheir. (2023). Smart Elevator Systems. Journal of Mechanical Materials and Mechanics Research. 6. 10.30564/jmmmr.v6i1.5503.

[10] Robal, Tarmo & Reinsalu, Uljana & Leier, Mairo. (2020). Towards Personalized Elevator Travel with Smart Elevator System. Baltic Journal of Modern Computing. 8. 10.22364/bjmc.2020.8.4.12.

[11] B. Nath Saha and A. Senapati, "Long Short Term Memory (LSTM) based Deep Learning for Sentiment Analysis of English and Spanish Data," 2020 International Conference on Computational Performance Evaluation (ComPE), Shillong, India, 2020, pp. 442-446, doi: 10.1109/ComPE49325.2020.9200054.

[12] A. Pulver and S. Lyu, "LSTM with working memory," 2017 International Joint Conference on Neural Networks (IJCNN), Anchorage, AK, USA, 2017, pp. 845-851, doi: 10.1109/IJCNN.2017.7965940.

[13] E. Chondrodima, N. Pelekis, A. Pikrakis and Y. Theodoridis, "An Efficient LSTM Neural Network-Based Framework for Vessel Location Forecasting," in IEEE Transactions on Intelligent Transportation Systems, vol. 24, no. 5, pp. 4872-4888, May 2023, doi: 10.1109/TITS.2023.3247993.

[14] Y. Qiao, K. Xu and K. Zhou, "Research on time series based on improved LSTM," 2023 IEEE 3rd International Conference on Power, Electronics and Computer Applications (ICPECA), Shenyang, China, 2023, pp. 951-958, doi: 10.1109/ICPECA56706.2023.10076103.

[15] N. S. Talwandi, A. Tripathi and S. Khare, "An Automatic Prediction of Elevator Capacity Using ML to Customize the Total Capacity," 2024 IEEE International Conference on Computing, Power and Communication Technologies (IC2PCT), Greater Noida, India, 2024, pp. 1405-1410, doi: 10.1109/IC2PCT60090.2024.10486469.

[16] Syed, Fahad Iqbal & Alshamsi, Mohammed & Dahaghi, Amirmasoud & Neghabhan, Shahin. (2020). Artificial lift system optimization using machine learning applications. Petroleum. 8. 10.1016/j.petlm.2020.08.003.

[17] Mahdi, Mohaned Alhaj A., Mohamed Amish, and Gbenga Oluyemi. 2023. "An Artificial Lift Selection Approach Using Machine Learning: A Case Study in Sudan" Energies 16, no. 6: 2853. https://doi.org/10.3390/en16062853.

[18] Shi, M., & Choi, Y. (2024). Comparison of the elevator traffic flow prediction between the neural networks of CNN and LSTM. *Intelligent Control and System Engineering*, 2(1), 1871. https://doi.org/10.59400/icse1871.

[19] Cui, T., Wang, L., Shi, X., Wang, X., Deng, L., & Wang, Q. (2024). An elevator trapping fault warning method based on self-attention-LSTM. *IET Conference Proceedings*, 2024(12), 3575. https://doi.org/10.1049/icp.2024.3575.

[20] Cao, Z., Guo, R., Tuguinay, C. M., Pock, M., Gao, J., & Wang, Z. (2022). Application of Deep Q Learning with Simulation Results for Elevator Optimization. *arXiv preprint arXiv:2210.00065*. https://arxiv.org/abs/2210.00065.

[21] Ma, W., Gao, X., Shi, J., Hu, H., Wang, C., Liang, Y., & Karakus, O. (2022). Reinforcement Learning based Voice Interaction to Clear Path for Robots in Elevator Environment. *arXiv preprint arXiv:2203.09844*. https://arxiv.org/abs/2203.09844.

[22] Al-Kodmany, K. (2023). Smart Elevator Systems. *Journal of Mechanical Materials and Mechanics Research*, 6(1), 41–53. https://doi.org/10.30564/jmmmr.v6i1.5503.

[23] Rashed, A. N. Z., Yarrarapu, M., Prabu, R. T., Antony, G. S. R., Edeswaran, L., Kumar, E. S., Aswitha, K., Snehith, N., & Hasane Ahammad, S. (2024). Connected smart elevator systems for smart power and time saving. *Scientific Reports*, 14, 19330. https://doi.org/10.1038/s41598-024-69173-1.

[24] Cao, Z., Guo, R., Tuguinay, C. M., Pock, M., Gao, J., & Wang, Z. (2022). Application of Deep Q Learning with Simulation Results for Elevator Optimization. *arXiv preprint arXiv:2210.00065*. https://arxiv.org/abs/2210.00065.

[25] Ma, W., Gao, X., Shi, J., Hu, H., Wang, C., Liang, Y., & Karakus, O. (2022). Reinforcement Learning based Voice Interaction to Clear Path for Robots in Elevator Environment. *arXiv preprint arXiv:2203.09844*. https://arxiv.org/abs/2203.09844.

[26] Shi, M., & Choi, Y. (2024). Comparison of the elevator traffic flow prediction between the neural networks of CNN and LSTM. *Intelligent Control and System Engineering*, 2(1), 1871. https://doi.org/10.59400/icse1871.

[27] Cui, T., Wang, L., Shi, X., Wang, X., Deng, L., & Wang, Q. (2024). An elevator trapping fault warning method based on self-attention-LSTM. *IET Conference Proceedings*, 2024(12), 3575. https://doi.org/10.1049/icp.2024.3575.

[28] Cao, Z., Guo, R., Tuguinay, C. M., Pock, M., Gao, J., & Wang, Z. (2022). Application of Deep Q Learning with Simulation Results for Elevator Optimization. *arXiv preprint arXiv:2210.00065*. https://arxiv.org/abs/2210.00065.

[29] Ma, W., Gao, X., Shi, J., Hu, H., Wang, C., Liang, Y., & Karakus, O. (2022). Reinforcement Learning based Voice Interaction to Clear Path for Robots in Elevator Environment. *arXiv preprint arXiv:2203.09844*. https://arxiv.org/abs/2203.09844.

[30] Al-Kodmany, K. (2023). Smart Elevator Systems. *Journal of Mechanical Materials and Mechanics Research*, 6(1), 41–53. https://doi.org/10.30564/jmmmr.v6i1.5503.

[31] Rashed, A. N. Z., Yarrarapu, M., Prabu, R. T., Antony, G. S. R., Edeswaran, L., Kumar, E. S., Aswitha, K., Snehith, N., & Hasane Ahammad, S. (2024). Connected smart elevator systems for smart power and time saving. *Scientific Reports*, 14, 19330. https://doi.org/10.1038/s41598-024-69173-1.

[32] Cao, Z., Guo, R., Tuguinay, C. M., Pock, M., Gao, J., & Wang, Z. (2022). Application of Deep Q Learning with Simulation Results for Elevator Optimization. *arXiv preprint arXiv:2210.00065*. https://arxiv.org/abs/2210.00065.

[33] Ma, W., Gao, X., Shi, J., Hu, H., Wang, C., Liang, Y., & Karakus, O. (2022). Reinforcement Learning based Voice Interaction to Clear Path for Robots in Elevator Environment. *arXiv preprint arXiv:2203.09844*. https://arxiv.org/abs/2203.09844.

[34] Shi, M., & Choi, Y. (2024). Comparison of the elevator traffic flow prediction between the neural networks of CNN and LSTM. *Intelligent Control and System Engineering*, 2 ::contentReference[oaicite:0]index=0

[35] Wei, Q., Wang, L., Liu, Y., & Polycarpou, M. M. (2020). Optimal elevator group control via deep asynchronous actor-critic learning. *IEEE Transactions on Neural Networks and Learning Systems*, 31(5), 1532–1543. https://doi.org/10.1109/TNNLS.2020.2965208.

[36] Shi, M., Xu, X., & Choi, Y. (2024). Gaussian analysis of the elevator traffic under the typical office building. *Frontiers in Computing and Intelligent Systems*, 8(1), 75–82. https://doi.org/10.54097/379xzj23.

[37] Cao, Z., Guo, R., Tuguinay, C. M., Pock, M., Gao, J., & Wang, Z. (2022). Application of deep Q learning with simulation results for elevator optimization. *arXiv preprint arXiv:2210.00065*. https://arxiv.org/abs/2210.00065.

[38] Zhang, J., Tsiligkaridis, A., Taguchi, H., Raghunathan, A., & Nikovski, D. (2022). Transformer networks for predictive group elevator control. *arXiv preprint arXiv:2208.08948*. https://arxiv.org/abs/2208.08948.

[39] Aldahiyat, H. (2009). Graphical simulation of an elevator. *MATLAB Central File Exchange*. Retrieved from https://www.mathworks.com/matlabcentral/fileexchange/23156-graphical-simulation-of-an-elevator.

[40] Tan, S. K. (2016). Man in elevator simulation. *Physics Lens*. Retrieved from https://www.physicslens.com/man-in-elevator-simulation/.

[41] Batosalem, A. C., Gaba, J. M. B., Ngo, J. B. O., Ticug, J. R. G., Ngo, C. A. M., & Ngo, J. B. O. (2019). Passenger flow simulation to optimize elevator traffic. *AnyLogic Simulation Software*. Retrieved from https://www.anylogic.com/resources/articles/passenger-flow-simulation-to-optimize-elevator-traffic/.

[42] Aldahiyat, H. (2009). Graphical simulation of an elevator. *MATLAB Central File Exchange*. Retrieved from https://www.mathworks.com/matlabcentral/fileexchange/23156-graphical-simulation-of-an-elevator.

[43] Tan, S. K. (2016). Man in elevator simulation. *Physics Lens*. Retrieved from https://www.physicslens.com/man-in-elevator-simulation/.

[44] Batosalem, A. C., Gaba, J. M. B., Ngo, J. B. O., Ticug, J. R. G., Ngo, C. A. M., & Ngo, J. B. O. (2019). Passenger flow simulation to optimize elevator traffic. *AnyLogic Simulation Software*. Retrieved from https://www.anylogic.com/resources/articles/passenger-flow-simulation-to-optimize-elevator-traffic/.